

Small-Model Adaptation on a T4: LoRA Quality and Inference Performance of Pythia-410M

Yash Dave

Abstract

We train Pythia-410M on a selected combination of 11 text datasets with LoRA and evaluate its performance and inference costs on a Colab Tesla T4. We explore three primary settings that contrast adapters with 8 and 16 ranks and learning rate values of 10^{-4} and 5×10^{-4} . The best performing validated adapter with 16 ranks decreases the test perplexity from 23.1134 to 19.6587 (14.95%) in 244,760 prediction targets with enhancements for all 11 texts.

1. Methodology and data

Pythia is an autoregressive GPT-NeoX model [1]. We use EleutherAI/pythia-410m. The model has 24 layers, hidden size 1,024, 16 attention heads, a 4,096-dimensional feed-forward neural network and a 2,048-token context capacity. The base has approximately 405.3 million parameters. Training makes use of 256-token chunks.

Dataset includes WikiText-2 raw, TinyStories, AG News, IMDb, Yelp Review Full, Amazon Polarity, CNN/DailyMail 3.0.0, XSum, SQuAD contexts, Emotion, and Rotten Tomatoes. Only natural language fields are utilized: articles/docs for news, context for SQuAD, title plus content for Amazon, and text everywhere else. No labels and Q&A targets are utilized. These are samples drawn from Hugging Face datasets. ID's, subset, and version of sources are provided in the repository configuration and data manifest.

Normalization of Unicode and whitespaces, preservation of 40–20,000 character texts, and canonical text deduplication are required preprocessing steps. Allocation by seed hashing has the probability of 80/10/10 but preserved after source post-processing are 1,000/100/100 texts correspondingly. Thus, preserved ratios differ from 80/10/10. Bounded streaming shuffle is practical sampling and not uniform sampling of all the sources. Data sets for reasoning were not chosen, but contents are not filtered semantically for all incidental reasoning content.

Split	Documents	Chunks	Prediction targets
Train	11,000	16,381	2,467,818
Validation	1,100	1,634	246,145
Test	1,100	1,629	244,760

Tokenization retains entire documents and adds EOS tokens. Length 256 chunking allows for overlap of one context token and ensures that each next-token target is retained precisely once without concatenation of documents. Padding tokens are masked according to position while EOS targets are retained. Manifests contain checksums. Exact deduplication does not exclude near duplicates.

2. LoRA training and model selection

LoRA learns low-rank updates while base weights remain fixed [2]. We target each layer's fused query/key/value projection, with LoRA dropout 0.05 and no trainable bias.

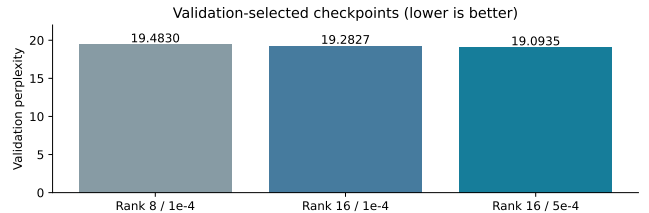
For rank 8 $\alpha = 16$ and 786,432 trainable parameters, and for rank 16, $\alpha = 32$ and 1,572,864 trainable parameters. Hence, the ratio $\alpha/r = 2$ is constant.

In all main experiments, AdamW optimizer, weight decay 0.01, gradient clipping 1.0, seed 42, batch size 2 with accumulation 8 (16 chunks per step) were used. The auto-mix precision in FP16 was applied to FP32 weights with gradient checkpointing. Cosine learning rate scheduling with 154 warmup steps (ceiling of $0.05 * 3072$). Common limitation is 3,072 steps that correspond to about three epochs. The evaluation and checkpoint saving occur after each 100th step and at the end.

The early stopping allows up to five consecutive validations without improvements by 0.001 in validation token loss after the last meaningful best value.

Absolute best-checkpoint selection is performed separately, so that the run could stop early and still make small improvements. Rank 16 at 10^{-4} stopped early at 3,000 steps, the other runs reached the limit. Colab interruptions have been restored using persistent checkpoints. Each execution has its own W&B segment.

Configuration	LR	Stop step	Best step	Val. PPL
Pretrained baseline	—	—	—	22.3108
Rank 8	10^{-4}	3,072	2,800	19.4830
Rank 16	10^{-4}	3,000	3,000	19.2827
Rank 16	5×10^{-4}	3,072	3,072	19.0935



At rank 16, increasing peak learning rate from 10^{-4} to 5×10^{-4} improves validation perplexity by approximately 0.98%. This supports selection among the tested settings, not a global optimum.

The late plateau occurred under a decaying learning rate; this does not establish its cause or global convergence. The test at 300 steps with rank 8 is an independent schedule and is not included in budget calculations.

Token loss is summed negative log-likelihood, using natural logarithms, divided by valid prediction targets. Perplexity is

calculated as $\exp(\text{loss})$, bits/token is calculated as $\text{loss}/\ln 2$. Sequence loss is NLL summed over sequence length divided by the number of chunks. W&B logs training loss, throughput, schedule, gradient norm, configuration, checkpoints, and validation metrics, length buckets, examples, rankings.

Final numbers are recalculated from logged metrics instead of screenshots.

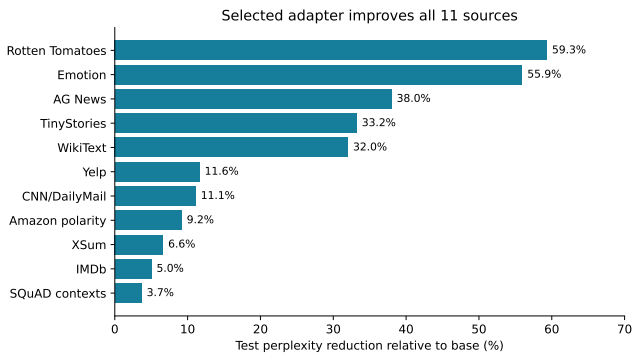
3. Held-out test results

Rank 16 is picked at 5×10^{-4} , step 3,072, through validation only and was later compared against the base on the test split. This process makes use of the same targets that have maximum sequence length of 256, batch size 2 and FP16 autocast. No other adapter gets tuned on the test set.

Metric	Base	Selected adapter
Token loss	3.140413	2.978518
Perplexity	23.113408	19.658658
Bits/token	4.530658	4.297093
Sequence loss	471.852339	447.527333

Perplexity reduction by the test set is **14.95%**. All 11 datasets show improvement, but there is a large variation in how much. This is perplexity for next-token language modeling, which applies to even corpora created for classification tasks.

Source	Targets	Base PPL	Adapter PPL	Red. (%)
WikiText	13,848	30.595	20.801	32.01
TinyStories	23,485	8.328	5.562	33.22
AG News	6,176	36.090	22.375	38.00
IMDb	28,411	35.365	33.585	5.03
Yelp	18,117	29.284	25.875	11.64
Amazon polarity	9,943	37.515	34.071	9.18
CNN/DailyMail	77,765	22.232	19.762	11.11
XSum	46,780	19.918	18.598	6.63
SQuAD contexts	15,317	20.870	20.102	3.68
Emotion	2,323	103.194	45.466	55.94
Rotten Tomatoes	2,595	117.648	47.846	59.33

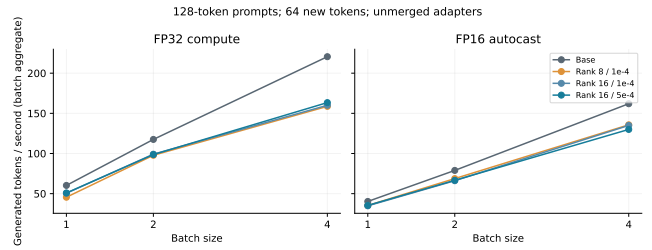


Perplexity is calculated using token-based loss rather than the mean of source perplexities. CNN/DailyMail and XSum make up 50.89% of test targets even though both have an equal document limit per source. Some short-text datasets show larger relative improvements, but the experiment does not isolate the effects of text content and length.

4. Inference benchmark protocol and results

It consists of four versions (base and three best adapters) with batches 1/2/4, prompt lengths 64/128/256, and FP32 computation versus FP16 autocast – 72 combinations. All 72 combinations are warmed up twice and repeated five times, yielding 360 runs. The sequences always produce 64 tokens using greedy decoding with EOS termination switched off. Prompts are not generated based on the tests but validation data. Four validation documents are reused across prompt lengths.

Weights continue to be FP32 in both precisions modes. Adapters are *unmerged*; SDPA and KV caching are enabled while TF32 is disabled. GPU-synchronized wall-clock latency measures full batch generation, including prefill and decoding. Model loading, tokenization, transfer, warming up, and logging are not included. The throughput is computed as the number of tokens generated over the total timing seconds, and not the prompt tokens. The time to first token was not recorded.

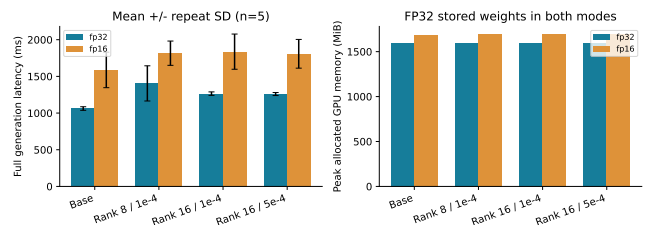


For length 128, the throughput of the FP32 base model is between 60.27 tokens per second and 220.53 tokens per second for a batch size of 1 and 4, respectively. The throughput of the chosen adapter is from 50.82 tokens per second to 163.36 tokens per second. This is the total throughput since more tokens are generated simultaneously.

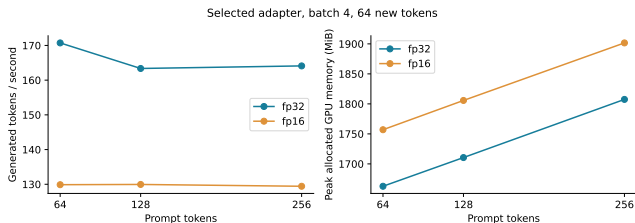
Variant	Mode	Batch	Lat. (ms)	Tok/s
Base	FP32	1	1,062.0	60.27
Selected	FP32	1	1,259.4	50.82
Base	FP16 autocast	1	1,588.1	40.30
Selected	FP16 autocast	1	1,808.0	35.40
Base	FP32	4	1,160.8	220.53
Selected	FP32	4	1,567.1	163.36

AutoCast FP16 performs worse in this case. Possible reasons include the casting itself and overhead associated with small workload start-up; however, they are not decoupled in the current test suite. The weights themselves are FP32; therefore, this test can't measure full-loaded FP16 vs. FP32 models. Rank 8 is not necessarily faster than rank 16; further testing should demonstrate how rank affects performance.

5. Latency, memory and context-length effects



When the batch size is 1 and prompt length is 128, the maximum amount of memory allocated by the selected adapter will be approximately 1,598.8 MiB in FP32 mode and 1,695.9 MiB in FP16 mode while applying autocast. FP32 parameters and possible temporary casts can affect allocation, but this benchmark does not isolate the causes of the memory difference. Memory denotes peak PyTorch allocation, not total device memory. Error bars show sample standard deviation across five timing repetitions.



Length comparisons offer various inference prompts while providing consistent output length. They are *not* comparison of models with different types, which are trained by using different sequence lengths. No sequence-1024 training experiment was performed; the unused template was removed. The mean values for each test, alongside repetitions and max allocation/reservation values, are given in the respective JSON/CSV files.

6. Discussion and limitations

LoRA allows for a positive adaptation with fewer than 0.4% trainable parameters in this experiment. Rank 16 and the higher learning rate enable enhanced validation performance by a little bit; the chosen model shows better test performance. Experiments use one seed, without measuring variability across training seeds. Insufficient run time, early stopping, data sample size and cosine decay preclude conclusions regarding convergence. No evidence of numerical instability or memory problems in the submitted results is reported; run-time disconnections do not constitute a failed optimization configuration.

Unmerged adapters introduce additional inference steps. The merging of adapters and weight loading in reduced precision may affect adapter performance in practice; however, no comparison of the above approaches was conducted. Only five iterations, fixed order of execution, same hardware and four benchmark prompts used for evaluation restrict generalizability of this experiment.

7. Reproducibility and artifact inventory

Final evaluation and benchmarking record Python 3.13.15, PyTorch 2.11.0+cu128, Transformers 4.57.6, PEFT 0.18.1, W&B 0.28.1 and CUDA 12.8 on a Tesla T4.

- `results/final_test/`: original base/adapter scores, per-source and bucket metrics, selection specification and environment.
- `results/inference_benchmarks/`: complete 72-case summary, individual measurements, session and environment metadata.
- `scripts/analyze_results.py`: checks coverage, re-

peated measurements, derived metrics and consistency; regenerates tables and figures without a GPU.

- `notebooks/analysis.ipynb`: interactive entry point for the same analysis.
- `results/tables/verification.json`: hashes and consistency-check summary.
- `report/`: editable LaTeX, bibliography, figures and compiled PDF.

Run from the repository root:

```
python -m pip install -r requirements-analysis.txt
python scripts/analyze_results.py
```

Repository: [slm-lora-performance-analysis on GitHub](#).

Experiment project: [Weights & Biases project](#). Direct final runs: [test evaluation](#) and [inference benchmark](#). Training continuation runs: [rank 8](#), [rank 16](#), [lower LR](#), and [rank 16, higher LR](#). Resumed-run links may contain only part of a training history.

The editable Overleaf project is linked in the repository documentation.

Original training histories, resolved configurations, environment records, and validation results are archived under `results/exports-submission-20260923-050201/`.

All 205 artifact checksums in the export were verified.

References

- [1] Stella Biderman et al. Pythia: A suite for analyzing large language models across training and scaling. *arXiv preprint arXiv:2304.01373*, 2023.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.